

Video Link: https://mediaspace.msu.edu/media/ClassicAgentModule/1_k2na5uws

We designed this module around a world-driven architecture. To keep our code clean, we built a universal template called the ClassicAgent. Think of this as our basic survival starter kit. If you look here at our Sense function, this is the eyes of the operation. Every turn, it runs this nested 3x3 for-loop to scan the immediate grid. It looks to find walls, resources, and enemies. Then will actively updates our team's shared TileKnowledge. But it doesn't just look at the map; it also checks our bank. Down here, it queries the global inventory and pushes our exact wood, stone, steel, and wheat counts directly into the agent's Blackboard memory. Because this base class runs first, it ensures every single agent on the board shares the exact same vision and knows exactly what we can afford. This ensures that every agent on the map shares the exact same vision and knows exactly what we can afford to build.

But seeing the world is only half the battle; the agents actually need to navigate it without getting stuck. That's where our PathGenerator class comes in. You can think of this as the built-in GPS for our entire game. Instead of hardcoding movement into every single character, any agent can just hand a request to the PathGenerator. An agent can ask for the shortest route to a specific coordinate, or say 'take me to the nearest wood or stone,' and the generator calculates the safest path. In the 'Explore' routing, If an agent is idle, it asks the GPS where to go. The generator looks at our shared team map, finds the 'frontier', which is the exact edge between what we've discovered and the fog of war, and plots a path to push that boundary outward. We also built in hard safety caps, so even if the map gets huge, the pathfinder won't freeze or crash the game engine. By building these universal tools, the eyes in the base agent and the GPS in the path generator, we made building advanced characters much easier. I'll pass it to my teammate to show how our ClassicDynamicAgent puts all this together to win the game.

This is our ClassicDynamicAgent, which is the dynamic-world version of our original agent behavior. These agents respond to every world tick, which are caused by players movements such as UP, LEFT, RIGHT, DOWN. The goal here is to have the agents work together in a sort of shared-resource system. Most of the focus on collecting and exploring, while one designated builder agent uses those shared resources to construct structures. You can see here the agents are moving around, gathering materials like wood, stone, and wheat, and spreading across the map.

Now looking at the code, the agent is driven by a behavior-tree-style blackboard system, instead of the actual behavior tree system that the classic agent runs under. In the sense() function, the agent scans nearby tiles, updates shared knowledge, checks if it's standing on grass or on a resource, and then decides what action to take. That decision is stored as a string called chosen_action. Then in GetAction(), that string gets converted into a numeric action ID, which is what DynamicWorld actually uses to execute the actions.

For the builder agent specifically, that falls under a different part of the sense function. It checks the global resource count from DynamicWorld, and then chooses what to build based on a priority system. As the top is the win condition, which is building the townhall. Below that, it

prioritizes essential structures like quarries to generate steel, and then expands into farms, spawners, and lumberyards based on what resources are available. It also makes sure it's standing on a valid grass tile before attempting to build, since that is one of the constraints of the world.

So overall, this agent now, senses and shares map knowledge, collects resources efficiently, explores the map, avoids getting stuck in loops, and if it's the builder, makes decisions about when and what to build.

Transitioning to the EnemyAgent. Every tick, it basically runs a sense and act loop. The Sense function is where it gathers information about the world. It looks through nearby agents, finds the player based on the name, and then figures out two things using Manhattan distance: whether the player is within its vision radius, and whether the player is right next to it. Based on that, it sets the internal state like `player_in_vision` and `player_adjacent`, and it also precomputes things like a chase move or a patrol move. So the Sense function is really just updating what the agent knows about the world.

Then in `SelectAction`, it uses that information to decide what to do. If the player is adjacent, it attacks. If the player is in vision but not adjacent, it chases by moving one step closer.

Otherwise, it just patrols back and forth. So it's constantly switching between these states.

The EnemyAgent also has some basic health logic. It tracks current HP and max HP, and you can damage it with `TakeDamage`, which clamps the value so it never goes below zero. Then there's an `IsAlive` check, which just returns whether HP is greater than zero. So that part handles whether the enemy can keep acting or should be considered dead.

For the GoblinAgent, there are only a few important functions since it's a pretty simple, stationary agent. First, there is `SetBlocking` and `ClearBlocking`, which control whether the goblin is currently blocking the path. Then `IsBlocking` just returns that state. There's also `IsPlayerAdjacent`, which tells us if the player is next to the goblin, and `CanBePaid`, which combines both conditions to check if the goblin is blocking and the player is close enough to interact.

The most important logic happens in the Sense function. Each tick, it resets the `player_adjacent` flag, gets its own position, and then looks through nearby agents from the world. It filters for the player using the name, and once it finds them, it checks the Manhattan distance. If the player is exactly one tile away, it sets `player_adjacent` to true.

Finally, `SelectAction` just calls Sense and returns 0, so the goblin never moves or act. It just updates its internal state and lets the rest of the system handle interactions.

ActionMap is a string-to-function registry. Instead of hardcoding function calls throughout your program, you register functions to a string name and trigger them dynamically at runtime. The class exposes a clean public API.

`AddFunction` lets you register a new callable under a name, returning an error if that name is already taken. `ReplaceFunction` does the same but overwrites any existing entry, which is useful when you need to swap behavior at runtime. `Trigger` takes a name and arguments, and ActionMap looks up the function, validates that the argument types match using type-index checking, calls it, and catches any exceptions it throws, all wrapped in a `std::expected` so callers

always know whether it succeeded. The classes RemoveFunction, HasFunction, GetFunctionNames, Clear, and Count give full control over what's registered. Each registered function is stored as a `std::any` alongside a `std::type_index`. This lets the class support functions with any signature while still catching type mismatches at call time, giving you flexibility without sacrificing safety. Copy construction and assignment are deleted intentionally, since the registry is meant to be owned in one place. This is done by implementing AddFunction, ReplaceFunction, and Trigger naturally and with a template type to allow multiple-returning arguments.

Starting off our tests we have the DynamicTreeBuilderTest file. This is only testing the behavior tree logic for the Dynamic world so it goes through its many branches such as townhall, spawner, quarry, etc. and it tests if the tree will make the right decisions by setting materials to a certain amount, seeing if it does or does not have enough materials to build and if it does, is it allowed to be placed on a certain block. Moving on we have the Classic Agent test file that expands on Dynamic Tree Builder and includes sense and action calls for our agents. It mainly tests movement and resource sensing. For the heavy interaction world we have two test files. First is the Enemy agent class. In the heavy interaction world enemy agents patrol the world and when a user is sensed, it then chases them. So that is what enemy agent test tests for. It sets an agent and sees if it patrols and when an enemy is sensed, it chases. Furthermore, we have the goblin agent test which is another agent in the interaction heavy world. Goblin agents defend a position until they are paid off from the user and it is at that point when they move over. So goblin agent test sets the goblin, checks if it remains stationary, makes sure it senses the user agent properly and then checks if it is able to stop blocking their position when the user agent pays them.